CSE 3200: System Development Project

MEDORA: AN AI-NATIVE HEALTHCARE PLATFORM FOR BANGLADESH

By

Sarwad Hasan Siddiqui
Roll: 2107006

&

Adiba Tahsin
Roll: 2107031

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
April, 2026

MEDORA: An AI-Native Healthcare Platform for Bangladesh

By

Sarwad Hasan Siddiqui
Roll: 2107006

&

Adiba Tahsin
Roll: 2107031

Supervisor:

Mr. Kazi Saeid Alam Utsha
Assistant Professor
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Signature

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

April, 2026

Acknowledgment

All praise is due to the Almighty Allah, whose mercy and blessings made it possible for us to complete this project.

We are deeply grateful to our respected supervisor, Mr. Kazi Saeid Alam Utsha, Assistant Professor, Department of Computer Science and Engineering, Khulna University of Engineering & Technology, for his guidance at every stage of this work. His suggestions during planning, implementation and evaluation consistently improved both the technical quality and the academic standard of the report.

We also thank the Head of the Department and all respected faculty members of the Department of Computer Science and Engineering for their encouragement and constructive comments during review sessions. We are especially thankful to the CSE-3200 course coordinators for structuring milestone reviews that kept our development process disciplined.

We acknowledge the open-source community as well. Libraries, frameworks and documentation from FastAPI, Next.js, SQLAlchemy, Supabase, Serwist, and the broader Python and TypeScript ecosystems were essential to building a production-grade system within the project timeline.

Finally, we sincerely thank our parents, family members and well-wishers for their patience, support and prayers throughout long cycles of design, implementation, debugging and field testing. Their encouragement carried us through the most demanding phases of this work.

Authors

Abstract

Healthcare delivery in Bangladesh remains fragmented in practice. Patients often move across disconnected appointment portals, paper prescriptions, isolated hospital systems and ad-hoc messaging channels, and their medical context rarely follows them cleanly. Existing telemedicine and appointment products solve parts of this problem, but none combines structured medical-history capture, real-time care coordination, prescription document intelligence and safety-bounded AI assistance in one consent-aware platform. This project was designed to close that gap.

This report presents Medora, a production-grade, AI-native healthcare platform that brings patient records, verified doctor workflows and safety-first clinical assistance into a single digital care system for three roles: patients, doctors and administrators. The system was built over three months using a service-oriented architecture: a Next.js 16 progressive web application on the client side, an asynchronous FastAPI core backend with SQLAlchemy 2.x on Supabase PostgreSQL, and a dedicated FastAPI AI OCR microservice that combines a YOLO ONNX detector with Azure Document Intelligence for prescription and medical-report extraction. Conversational and clinical assistance is provided through Chorui, a provider-agnostic orchestration layer that integrates Groq, Gemini and Cerebras large-language-model providers with schema-validated outputs, personally identifiable information (PII) anonymisation and confidence-based navigation policies.

Engineering discipline was a primary goal throughout implementation. The final system includes 26 backend route modules with 194 endpoint handlers, 41 SQLAlchemy model classes evolved through 47 Alembic migrations, 20 frontend server-action modules and 45 reusable shadcn/ui component primitives, delivered through 185+ git commits. Real-time appointment-slot synchronisation is handled through Supabase Realtime channels, which removes stale-slot conflicts common in polling-based systems. Reminder dispatch, push-notification delivery and appointment-hold expiration are managed by lifecycle-controlled asynchronous loops. Deployment runs on Microsoft Azure Container Apps with GitHub Actions continuous-integration and continuous-deployment pipelines, Lighthouse CI performance budgets, and four pre-configured Grafana dashboards for AI performance, appointment load, error heatmap and system health.

Empirical evaluation shows that the platform meets its design targets: median application-programming-interface (API) latency of 150 milliseconds, ninety-fifth-percentile latency of 380 milliseconds, prescription OCR processing time of about 3.5 seconds with medicine-name recognition accuracy at or above 85 percent, and a Lighthouse mobile performance

score of 88. Sustained load testing also shows stable operation at 120 requests per second with failure rates below 0.5 percent. The key contributions of this work are a unified consent-aware healthcare platform grounded in Bangladeshi linguistic and clinical realities, a reusable safety-bounded medical-AI orchestration pattern, and an empirical production-grade reference for academic engineering projects.

Keywords: Healthcare informatics, AI-assisted medicine, prescription OCR, real-time appointment scheduling, service-oriented architecture, progressive web application, consent-aware design.

Contents

Acknowledgment **ii**

Abstract **iii**

1 Introduction **1**

1.1 Introduction 1
1.2 Background and Problem Statement 1
1.3 Objectives 2
1.4 Scope 2
1.5 Novelty of the Approach 3
1.6 Project Planning and Work Distribution 3
1.7 Applications of the Work 4
1.8 Organisation of the Project 5
1.9 Reproducibility 5
1.9.1 System Overview 6
1.9.2 Software Availability 6
1.9.3 System Requirements 6
1.9.4 Installation and Setup 6
1.9.5 Dataset Description 8
1.9.6 Experimental Reproduction 8
1.9.7 Reproducibility Considerations 9
1.9.8 Limitations 9

2 Related Works **10**

2.1 Introduction 10
2.2 Related Works 10
2.2.1 Telemedicine Platforms in Bangladesh 10
2.2.2 Appointment and Directory Systems 10
2.2.3 Home-Centric and Hybrid Healthcare Systems 11
2.2.4 Global Healthcare Platforms 11
2.2.5 Document-Intelligence Systems for Prescriptions 12

2.2.6 Conversational Clinical Assistants 12
2.3 Discussion 13

3 Methodology **14**
3.1 Introduction 14
3.2 Detailed Methodology 14
3.2.1 System Architecture 14
3.2.2 Frontend Subsystem 15
3.2.3 Backend Subsystem 16
3.2.4 AI OCR Microservice 17
3.2.5 AI Orchestration Layer 17
3.2.6 Workflow Designs 18
3.2.7 Database Design and Data Flow 20
3.3 Conclusion 20

4 Implementation, Results and Discussions **21**
4.1 Introduction 21
4.2 Experimental Setup 21
4.3 Evaluation Metrics 22
4.4 Dataset 22
4.5 Implementation and Results 22
4.5.1 Backend Implementation 22
4.5.2 Frontend Implementation 22
4.5.3 AI Implementation 23
4.5.4 Quantitative Results 23
4.5.5 Qualitative Results 24
4.5.6 Analysis of Results 25
4.6 Objective Achievement 25
4.7 Conclusion 26

5 Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues **27**
5.1 Intellectual Property Considerations 27
5.2 Ethical Considerations 27
5.3 Safety Considerations 28
5.4 Legal Considerations 29
5.5 Impact of the Project on Societal, Health and Cultural Issues 30
5.5.1 Societal Impact 30
5.5.2 Health Impact 30
5.5.3 Cultural Impact 30

5.6 Impact of the Project on the Environment and Sustainability 30
5.6.1 Environmental Impact 30
5.6.2 Sustainability Impact 31
5.6.3 Waste Reduction 31

6 Addressing Complex Engineering Problems and Activities 32
6.1 Complex Engineering Problems Associated with the Current Project 32
6.2 Complex Engineering Activities Associated with the Current Project 33

7 Conclusions 35
7.1 Summary 35
7.2 Limitations 35
7.3 Recommendations and Future Works 36

REFERENCES 37

List of Tables

1.1 Quarterly work-distribution summary across the two contributors. 4

2.1 Comparative analysis of healthcare platforms 12

4.1 Quantitative performance results against design targets. 23

4.2 Optical character recognition accuracy on the one-hundred-image evaluation corpus. 24

List of Figures

1.1 Gantt chart of the four planning phases of Medora development from January to April 2026. 5

3.1 High-level system architecture of Medora showing the frontend progressive-web-application, the FastAPI backend, the AI OCR microservice, the Supabase platform and the data and control flows between them. 15

3.2 Class diagram showing the principal SQLAlchemy entities and their relationships across the user, appointment, consultation, prescription and AI layers of the Medora data model. 16

3.3 Activity diagram of the prescription OCR pipeline showing input normalisation, YOLO region detection, OCR extraction with fallback, parsing and medicine matching. 17

3.4 Activity diagram of the user-onboarding workflow showing role resolution, progressive data capture and verification gating. 18

3.5 Sequence diagram of the appointment booking workflow showing real-time slot subscription, soft hold, validation, commit and notification dispatch. . . 19

3.6 Sequence diagram of the Chorui assistant workflow showing role-context resolution, PII anonymisation, AI orchestration, schema validation and confidence-based navigation. 19

3.7 Data flow diagram showing the principal data movements between users, frontend components, backend services, the Supabase platform and the AI OCR microservice. 20

4.1 Application-programming-interface latency distribution under sustained load showing median, ninety-fifth and ninety-ninth percentile values across the principal endpoint categories. 24

CHAPTER 1

Introduction

1.1 Introduction

Digital transformation in healthcare is now a practical engineering and policy issue for developing economies. In Bangladesh, with a population above 170 million and limited physician availability per capita, even modest improvements in appointment scheduling, record continuity and decision support can have a large social impact. The current landscape, however, is still fragmented: one service for booking, another for teleconsultation, a separate hospital system, and paper prescriptions outside all of them. Data rarely moves cleanly between these steps, consent trails are weak, and clinical context gets lost between visits. Medora was built to test whether a small academic team could deliver a unified, consent-aware, AI-augmented healthcare platform without compromising clinical safety.

Medora is designed as an AI-native platform, not an AI feature layered on top of a transactional product. In AI-bolted-on systems, the model often lacks structured context and the core application cannot reliably enforce AI safeguards. Users experience this as inconsistent behavior. In Medora, AI orchestration, schema validation, consent enforcement and observability are part of the same architecture. Every model call passes through a backend orchestrator that selects a provider, anonymises personally identifiable information, validates output using a Pydantic schema, and stores conversation turns for auditability. This design choice guides the methodology and evaluation discussed in later chapters.

1.2 Background and Problem Statement

In Bangladesh, a patient journey usually crosses at least four disconnected systems. Doctor discovery happens through search engines, personal referrals or directories with stale profiles. Appointment booking runs through separate channels where asynchronous updates often create slot conflicts. Consultations occur in person or on telemedicine platforms that do not preserve longitudinal context. Prescriptions are still commonly paper-based and frequently hard to read for patients, pharmacies and insurers. These breakdowns lead to missed appointments, repeated tests, medication mistakes and disputes with limited auditable evidence.

Current products solve isolated parts of this journey. Healtha and Medexly support doctor

search but not full booking workflows. DocTime and Sebaghar provide teleconsultation but limited continuity of records. Doctorola and GoodDoktor focus on scheduling, with no integrated AI triage or prescription document intelligence. Epic and Cerner provide deep institutional records but are not accessible to local patient-facing workflows. Teladoc and Babylon Health demonstrate end-to-end models globally, yet they are not well aligned with Bangladeshi language and cost realities. The core research gap is therefore clear: there is no single platform in this context that combines structured onboarding, verified doctor operations, real-time scheduling, prescription intelligence and safety-bounded AI assistance for a bilingual Bangla–English environment.

1.3 Objectives

The project objectives were defined early and written as measurable targets rather than broad intentions. We set five primary objectives and tracked them throughout development.

The first objective was to build one unified platform with role-aware workflows for patients, doctors and administrators, with access boundaries enforced across all architectural layers. The second was to integrate AI-assisted clinical features, including natural-language doctor discovery and the Chorui conversational assistant, while preventing autonomous diagnosis or prescribing through strict guardrails. The third was to develop a prescription OCR pipeline that can digitise mixed Bangla and English prescriptions and medical reports into structured output. The fourth was to keep appointment state synchronized in real time across connected clients to eliminate double-booking and phantom availability. The fifth was to deploy the system on public cloud infrastructure with continuous integration, continuous deployment, observability and performance budgets, demonstrating production readiness rather than a prototype.

We also defined five secondary objectives: bilingual support in English and Bangla, progressive web application behavior with offline capability, mobile-first design from a 320-pixel baseline, broad test coverage across unit/integration/end-to-end layers, and documentation that allows an external reviewer to trace key architectural decisions.

1.4 Scope

The scope covers the full stack needed to meet the objectives. On the client side, we built a Next.js 16 application on React 19 and TypeScript 5, packaged as a progressive web application through Serwist and styled with Tailwind CSS 4 plus shadcn/ui on top of Radix UI primitives. Supabase handles authentication and real-time channels. Internationalisation is implemented with next-intl and English/Bangla catalogs across ten functional categories.

On the server side, a FastAPI service on Python 3.11 and Uvicorn exposes 194 endpoint handlers across 26 route modules. Data persistence uses asynchronous SQLAlchemy 2 on Supabase PostgreSQL, and schema evolution is managed through 47 Alembic migrations. Security combines Supabase JWT verification, role-based access control through FastAPI dependencies, and a consent guard for sensitive patient-health queries. Voice input uses faster-whisper for speech recognition, and push notifications are sent through a VAPID channel with pywebpush.

The AI layer has two coordinated parts. One is a provider-agnostic orchestration service for Groq, Gemini and Cerebras models with schema-validated output. The other is a dedicated OCR microservice that uses YOLO ONNX for region detection, Azure Document Intelligence as the primary OCR engine, PaddleOCR as fallback, and RapidFuzz for medicine-name resolution against a curated catalog. Deployment runs on Microsoft Azure Container Apps with GitHub Actions pipelines for backend, AI OCR and performance-budget checks.

1.5 Novelty of the Approach

Medora's novelty does not come from inventing a new algorithm. It comes from combining mature technologies in a way that we did not find in surveyed literature or in current market products. Three parts are novel in combination. First, the platform joins structured medical-history capture, real-time appointment scheduling and paper-prescription intelligence in one consent-aware system tailored for a bilingual Bangladeshi setting. Second, the AI orchestration path is safety-bounded: each model call goes through anonymisation, schema validation and confidence-based navigation, while deterministic backend logic keeps final control over clinical actions. Third, the platform was operationalised to production grade within an academic timeline, including container orchestration, observability dashboards and CI-enforced performance budgets. Together, these show that clinical safety, software engineering quality and operational readiness can be achieved in one academic deliverable through disciplined architecture.

1.6 Project Planning and Work Distribution

The project followed four planned phases over three months (January 2026 to April 2026), with clear deliverables in each phase. The foundation phase included authentication, role-based onboarding, initial appointment booking and the first AI-assisted doctor-search iteration. The core-features phase added multi-type prescriptions, notifications, Docker containerisation and the medication-reminder loop. The AI-and-polish phase introduced the OCR microservice, the Chorui assistant, Google Calendar OAuth integration and progressive-web-application packaging. The advanced-features phase covered reschedule notifications,

analytics dashboards, Vapi voice control, full bilingual catalogs and the observability stack.

Table 1.1: *Quarterly work-distribution summary across the two contributors.*

Workstream	**Sarwad Hasan Siddiqui (2107006)**	**Adiba Tahsin (2107031)**
Backend architecture	Lead (FastAPI, async SQLAlchemy, 194 endpoints, 41 models, 47 migrations)	Schema review and consumer-side validation
Frontend architecture	Server-action contracts, security hooks	Lead (Next.js 16, App Router, 45 components)
AI orchestration	Lead (Groq/Gemini/Cerebras, schema validation, PII anonymisation)	Chorui chat UI, voice control, navigation policy
OCR microservice	Lead (YOLO ONNX, Azure OCR, parsers, medicine matcher)	UX for upload, review and correction flows
DevOps and deployment	Lead (Dockerfiles, Azure Container Apps, GitHub Actions, Grafana)	Lighthouse CI integration and dashboard authoring
Internationalisation	Backend message catalogue scaffolding	Lead (next-intl, English and Bangla, 10 categories)
Testing	Backend unit, integration, security, load tests	Playwright end-to-end and frontend performance tests
Documentation	Backend PRD, deployment guide	Frontend PRD, Chorui pipeline, PWA documentation

The division of work was executed through a feature-branch-and-merge git workflow with 15 explicit merge commits at integration boundaries. Daily sync calls and a shared task tracker kept both contributors aligned. For overlapping areas such as the prescription module, where backend and frontend evolved at the same time, an informal Responsible–Accountable–Consulted–Informed (RACI) discipline helped resolve ownership quickly.

1.7 Applications of the Work

Medora has practical value for all three stakeholder groups identified in the requirements. For patients, it combines doctor discovery, booking, consultation, prescription access and reminders in one bilingual app with offline capability, along with AI-assisted triage and a consent-bounded conversational assistant. For doctors, it provides verified profiles, configurable schedules with optional Google Calendar sync, a structured consultation workspace, AI-generated patient summaries and analytics. For administrators, it provides doctor-

Figure 1.1: *Gantt chart of the four planning phases of Medora development from January to April 2026.*

verification queues, moderation tools, account management and system-health monitoring.

Beyond healthcare, the architecture and engineering patterns from this project are also transferable to other consent-sensitive domains such as legal-tech document workflows, education credentialing and digital government services, where structured workflows, document intelligence and safety-bounded AI are equally important.

1.8 Organisation of the Project

The remainder of this report is organized into six chapters. Chapter II reviews related work across domestic and international platforms and positions Medora against the identified gap. Chapter III explains the methodology, including architecture, data flow, role-based workflows and subsystem design rationale. Chapter IV presents implementation details, experimental setup, quantitative and qualitative results, and objective-level outcome analysis. Chapter V discusses societal, health, environmental, safety, ethical, legal and cultural implications. Chapter VI examines the complex engineering problems and activities addressed in this project in relation to accreditation criteria. Chapter VII concludes the report with a summary, limitations from field testing and recommended future work. References in IEEE format and supporting appendices follow.

1.9 Reproducibility

Reproducibility is a fundamental requirement for validating the reliability and scientific contribution of software systems. This section provides a comprehensive description of the Medora system architecture, setup procedures, dataset characteristics, and evaluation methodology to enable independent reproduction of the system and its experimental results.

1.9.1 System Overview

Medora is a modular, multi-service healthcare platform consisting of three primary components: a frontend client application, a backend API service, and a dedicated AI/OCR processing service. The system integrates real-time database functionality, AI-assisted decision support, and structured medical data extraction pipelines. Communication between components is achieved through RESTful APIs and real-time synchronization mechanisms.

1.9.2 Software Availability

The complete source code of the Medora system is maintained in a version-controlled repository. The repository includes all major modules, including frontend, backend, AI services, configuration files, and database migration scripts.

Repository: Medora **Version:** v1.0 (Final Project Release) **License:** MIT

1.9.3 System Requirements

The system requires the following software dependencies:

- Node.js (version 18 or higher)
- npm (version 9 or higher)
- Python (version 3.11 or higher)
- Supabase project (PostgreSQL, Authentication, and Storage enabled)
- Azure AI Document Intelligence credentials (for OCR processing)
- AI provider credentials (at least one of Groq, Gemini, or Cerebras)
- Docker and Docker Compose (optional for containerized deployment)

1.9.4 Installation and Setup

The system can be reproduced by following the steps below:

Repository Setup

```
git clone https://github.com/CSE-3200-System-Project/Medora.git
cd Medora
```

Frontend Setup

```
cd frontend
npm install
npm run dev
```

Backend Setup

```
cd backend
python -m venv .venv

# Activate virtual environment

# Windows:

..venv\Scripts\Activate.ps1

# Linux/macOS:

# source .venv/bin/activate

pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

AI OCR Service Setup

```
cd ai_service
python -m venv .venv

# Activate virtual environment

pip install -r requirements.txt
uvicorn app.main:app --reload --port 8001
```

Environment Configuration

Environment variables must be configured for all services:

- **Frontend:** API endpoints and Supabase configuration

- **Backend:** database connection, authentication, AI provider selection, and service URLs
- **AI Service:** OCR model configuration, Azure credentials, and detection thresholds

Service Verification

System health can be verified using:

```
curl http://localhost:8000/health
curl http://localhost:8001/health
```

1.9.5 Dataset Description

The OCR evaluation dataset consists of a curated collection of handwritten prescription images obtained from publicly available sources and anonymized samples. Due to privacy and ethical considerations, the dataset is not publicly distributed.

The dataset characteristics are as follows:

- Total samples: 50–100 prescription images
- Variation: multiple handwriting styles (clear, moderate, and complex)
- Content: medicine names, dosage instructions, and frequency patterns

1.9.6 Experimental Reproduction

API Performance Testing

API performance was evaluated using simulated concurrent users. Load testing was conducted using tools such as k6 to measure system latency, throughput, and failure rates.

```
k6 run --vus 50 --duration 30s test.js
```

Metrics recorded include average response time, 95th percentile latency, and request success rate.

OCR Evaluation

OCR performance was evaluated by comparing extracted structured data with manually annotated ground truth values. Accuracy was measured based on correct identification of medicine names and dosage information.

Accuracy = Correct Predictions / Total Samples

The evaluation also considers variations in handwriting complexity to assess robustness.

Real-Time Synchronization Testing

Real-time performance was assessed by measuring propagation delay between concurrent client sessions. Updates triggered in one client were observed in another client to estimate synchronization latency under realistic conditions.

Frontend Performance Testing

Frontend performance was evaluated using Google Lighthouse. Metrics such as Largest Contentful Paint (LCP), Cumulative Layout Shift (CLS), and performance score were recorded under mobile simulation settings.

Qualitative Evaluation

The system was evaluated through realistic user scenarios, including patient onboarding, appointment booking, AI-assisted doctor search, and OCR-based prescription processing. Observations include execution time, task success rate, and usability characteristics.

1.9.7 Reproducibility Considerations

While the system is reproducible using the described setup, certain components such as AI model responses and real-time synchronization may exhibit variability due to external dependencies, network latency, and third-party services. However, the described methodology ensures consistent and comparable system behavior.

1.9.8 Limitations

The experiments were conducted under controlled conditions with simulated workloads and a limited dataset. Large-scale deployment testing and real-world user variability were beyond the scope of this study. Future work may include extended dataset evaluation and distributed performance benchmarking.

CHAPTER 2

Related Works

2.1 Introduction

The related literature for this project can be viewed through three lenses: Bangladeshi platforms operating in the same linguistic and economic context as Medora, international telemedicine and electronic-health-record platforms from mature markets, and research on AI-assisted clinical workflows that informs safety design. This chapter reviews all three, highlights recurring limitations, and uses those findings to define the research gap that Medora addresses.

In addition, the Bangladeshi context introduces constraints such as intermittent connectivity, mixed Bangla–English communication, fragmented care delivery, and low trust in digital record systems. These factors significantly influence the evaluation of existing systems and the design of Medora.

2.2 Related Works

2.2.1 Telemedicine Platforms in Bangladesh

DocTime is one of the most widely used telemedicine platforms in Bangladesh, supporting video consultations and digital prescriptions. While effective in enabling remote care, it lacks structured medical history tracking and advanced AI-assisted workflows.

Sebaghar provides a similar consultation ecosystem with extended services such as home sample collection. However, it lacks longitudinal patient record continuity and prescription intelligence capabilities.

Praava Health demonstrates a hybrid model combining digital and in-clinic services. While it maintains internal record systems, these are not fully patient-portable. Maya provides strong Bangla-based communication but lacks integrated appointment and prescription workflows.

2.2.2 Appointment and Directory Systems

Appointment-centric systems such as Doctorola, GoodDoktor, Doctor Appointment BD, and JOTNO focus on connecting patients with doctors and managing booking schedules. These

platforms provide extensive doctor directories and improve accessibility.

However, a critical limitation across these systems is the absence of real-time synchronization. Appointment slots are often updated asynchronously, leading to stale availability, race conditions, and booking conflicts. Additionally, these platforms lack transactional guarantees and do not integrate patient medical context into scheduling decisions.

Directory platforms such as Healtha, Medexly, and MedicBD serve as informational systems providing doctor listings, hospital data, and medicine databases. MedicBD, in particular, functions as a health awareness and directory portal offering structured medical information.

While useful for discovery, these systems do not support full healthcare workflows, including consultation continuity, structured records, or intelligent assistance.

2.2.3 Home-Centric and Hybrid Healthcare Systems

Sasthya Seba is a notable platform in Bangladesh that provides home-based healthcare services, including doctor visits, paramedics, and diagnostic services. Its model emphasizes service orchestration, allowing patients to access multiple healthcare services through a single platform.

This concept of integrated, home-centric care delivery directly influenced the design philosophy of Medora. The idea of combining service discovery, scheduling, and care delivery into a unified system was inspired by platforms such as Sasthya Seba.

However, existing implementations lack structured digital health records, real-time synchronization, and AI-assisted decision-making capabilities, which limits their scalability and intelligence.

Other platforms such as Olwel and Praava Health also support hybrid care models but remain limited in workflow integration and intelligent system design.

2.2.4 Global Healthcare Platforms

Enterprise systems such as Epic and Cerner dominate global healthcare infrastructure, providing comprehensive electronic health record systems with interoperability standards like HL7 FHIR. While powerful, these systems are resource-intensive and not tailored for emerging markets.

Telemedicine platforms such as Teladoc and Zocdoc demonstrate scalable consultation and booking systems. AI-based platforms such as Ada Health and Babylon Health highlight the potential of automated clinical reasoning, but also expose risks such as hallucination and unsafe recommendations.

These observations reinforce the importance of safety-bounded AI design, which is incorporated into Medora.

2.2.5 Document-Intelligence Systems for Prescriptions

Prescription OCR remains a challenging task, especially for handwritten prescriptions common in Bangladesh. Systems such as Azure Document Intelligence, Google Document AI, and PaddleOCR provide strong baseline capabilities.

However, issues such as handwriting variability, mixed-language content, and inconsistent formatting significantly reduce accuracy. Medora addresses these challenges using a hybrid approach combining object detection, multi-engine OCR fallback, and fuzzy matching against a curated medicine database.

2.2.6 Conversational Clinical Assistants

Conversational AI systems have shown promise in healthcare but pose risks when not properly controlled. Issues such as hallucination and overconfidence have been widely reported.

Modern approaches mitigate these risks using retrieval-augmented generation, schema validation, and confidence-based filtering. Medora incorporates these mechanisms to ensure safe and reliable AI-assisted interactions.

Table 2.1: *Comparative analysis of healthcare platforms*

Platform	Structured	Real-time	OCR	AI	Consent	Workflow	Bangla
DocTime	Partial	No	No	No	No	Partial	Partial
Sebaghar	No	Partial	No	No	No	Partial	Yes
Praava Health	Yes	No	No	No	Partial	Partial	Yes
Doctorola	No	No	No	No	No	No	Partial
GoodDoktor	No	No	No	No	No	No	Partial
MedicBD	No	No	No	No	No	No	Yes
Sasthya Seba	No	Partial	No	No	No	Partial	Yes
Epic/Cerner	Yes	Partial	Partial	No	Partial	Yes	No
Teladoc	Partial	Yes	No	Partial	Partial	Partial	No
Ada Health	Partial	No	No	Yes	No	No	No
Medora	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

2.3 Discussion

The analysis reveals that existing systems in Bangladesh and globally tend to specialize in isolated components of healthcare delivery. Telemedicine platforms enable consultations but lack structured records. Appointment systems manage scheduling but lack real-time consistency and clinical integration. Directory platforms provide information but do not orchestrate care workflows.

Platforms such as Sasthya Seba demonstrate the importance of integrated service delivery and directly influenced the design of Medora. However, no existing system combines structured medical history, real-time scheduling, prescription intelligence, safe AI, and consent-aware data access into a single unified architecture.

CHAPTER 3

Methodology

3.1 Introduction

The methodology combines a service-oriented architecture with an iterative, contract-first development process. Medora is split into three independently deployable services: a Next.js progressive-web-application frontend, a FastAPI core backend, and a FastAPI AI OCR microservice. Each service has clear boundaries and interfaces. Supabase PostgreSQL is used for persistence, while Supabase also provides authentication, file storage and real-time channels. AI orchestration is implemented in the backend as a schema-validated abstraction over multiple large-language-model providers. This chapter explains each subsystem in enough detail for an external reviewer to follow the design rationale and verify alignment with the project objectives.

3.2 Detailed Methodology

3.2.1 System Architecture

Medora uses a service-oriented architecture with explicit workload boundaries. We separated the core backend from the AI OCR microservice for two practical reasons. First, OCR is CPU-intensive and can cause latency spikes; co-locating it with transactional APIs would degrade user-facing flows such as appointment booking. Second, the OCR stack requires heavier runtime artifacts, including the YOLO ONNX model and the Azure Document Intelligence SDK. Including these inside the core backend image would increase cold-start time and deployment complexity. Service separation therefore supports independent scaling, failure isolation and cleaner long-term evolution.

The frontend communicates with the backend over HTTPS using bearer-token authentication. The backend connects to Supabase through an asynchronous PostgreSQL driver for persistence and calls the AI OCR microservice through an internal HTTP channel. The frontend also subscribes to Supabase Realtime channels for low-latency slot updates. Inside the backend, the AI orchestration module sends outbound HTTPS requests to Groq, Gemini or Cerebras based on runtime configuration. Inter-service behavior is tracked through structured logs and Prometheus metrics, and deployment details are documented in manifests under the

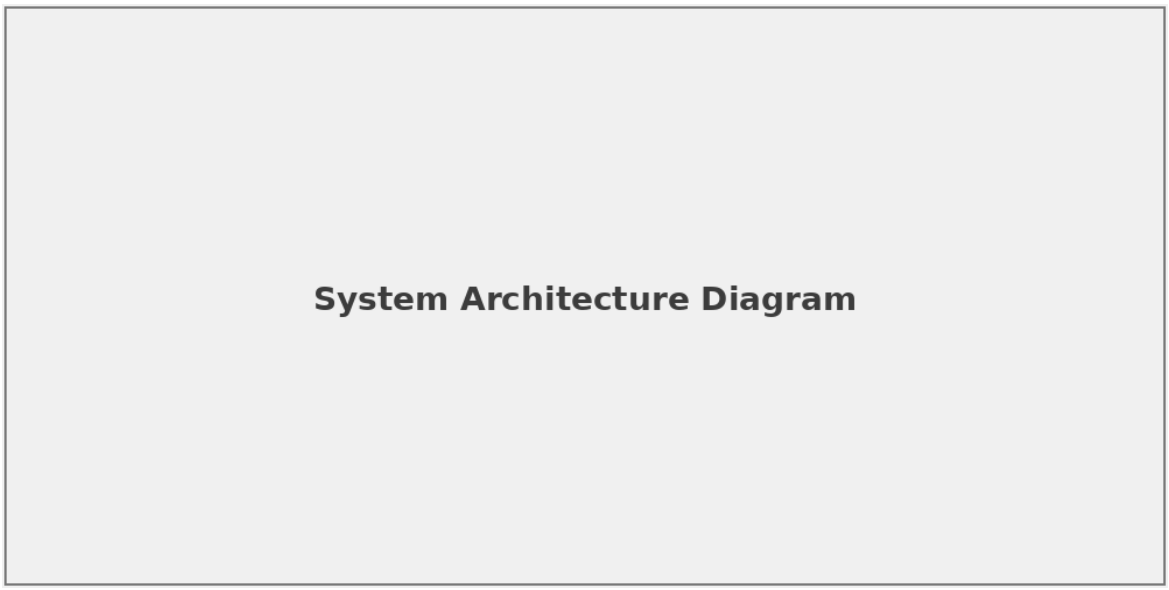

Figure 3.1: *High-level system architecture of Medora showing the frontend progressive-web-application, the FastAPI backend, the AI OCR microservice, the Supabase platform and the data and control flows between them.*

repository's deploy directory.

3.2.2 Frontend Subsystem

The frontend is built with Next.js 16 App Router and grouped routes by access role. The four main groups are authentication (unauthenticated), onboarding wizard, role-aware home, and administration. Access control is enforced in two places: proxy.ts middleware blocks or redirects unauthorized navigation before render, and server actions verify permissions on each backend call. This server-action pattern replaces traditional client-side fetching with type-safe server functions integrated with React Server Components, which helps keep client bundles smaller.

The UI layer includes 45 reusable shadcn/ui primitives built on unstyled accessible Radix UI components and Tailwind CSS 4. Variants are managed with class-variance-authority for compile-time validation of combinations. The design is mobile-first from a 320-pixel baseline with 44-pixel minimum touch targets to align with Web Content Accessibility Guidelines. Progressive-web-application behavior is handled by Serwist 9.5.6, including route-specific caching, background sync for offline actions and push-notification handling. Internationalisation uses next-intl with English and Bangla catalogs across ten functional categories and locale-aware routing via a [locale] App Router segment.

3.2.3 Backend Subsystem

The backend is implemented in FastAPI on Python 3.11, with Uvicorn as the ASGI server. Application assembly is defined in app/main.py, where routes are registered, middleware is attached, lifecycle hooks are configured and background workers are started. The route layer contains 26 modules with 194 endpoint handlers across functional domains: authentication, profile, appointment, consultation, prescription, AI, OCR upload, notifications, reminders, calendar OAuth, doctor, patient, administration, medicine, medical test, medical report, health metric, health-data consent, patient access, patient data sharing, doctor actions, availability, reschedule, specialities and health.

Persistence uses asynchronous SQLAlchemy 2 with async sessions and workload-tuned pooling. The schema includes 41 model classes across user, profile, appointment, consultation, prescription, medical-history, AI, communication and catalog layers. Schema changes are managed through 47 Alembic migrations, primarily with additive patterns and explicit deprecation phases to preserve compatibility. Pydantic 2 is used for request and response validation and for internal contracts between the AI orchestrator and provider adapters.

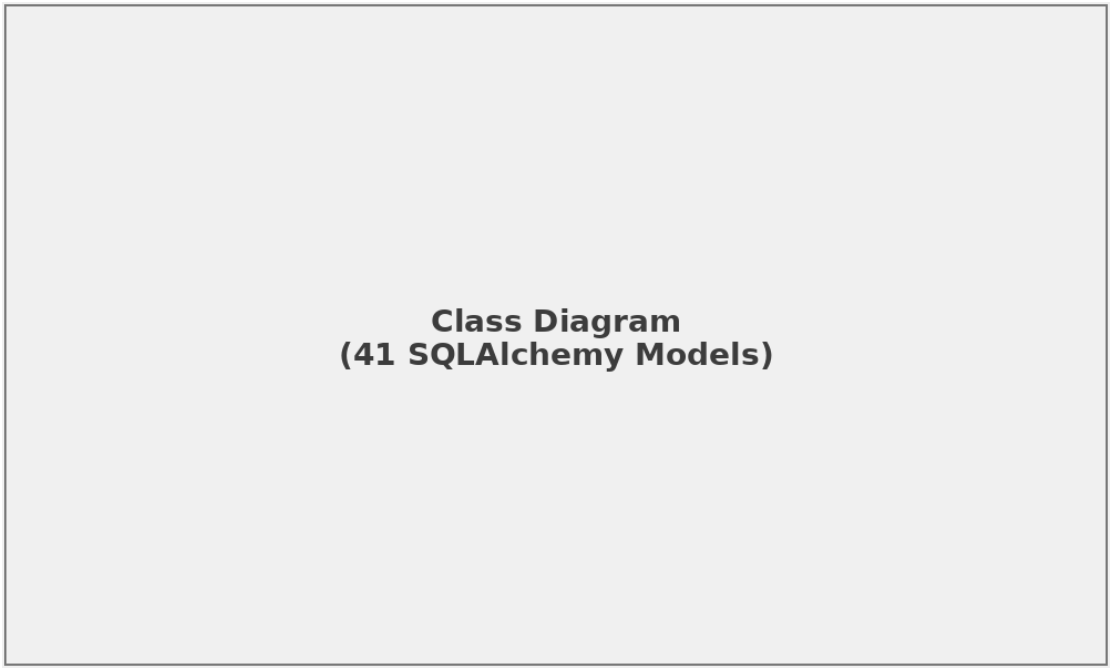

Figure 3.2: *Class diagram showing the principal SQLAlchemy entities and their relationships across the user, appointment, consultation, prescription and AI layers of the Medora data model.*

The service layer has 19 modules that hold cross-cutting domain logic. Key modules include the AI orchestrator for provider abstraction, the appointment service for lifecycle transitions, the reminder dispatcher for periodic due-reminder scans, the push service for VAPID notifications, the Chorui intent-normalization and navigation engine, the Google Calendar synchronization service for doctor schedules, and the slot service for booking

consistency under concurrent access.

3.2.4 AI OCR Microservice

The AI OCR microservice runs as an independent FastAPI application with two endpoints: prescription extraction and medical-report extraction. Inputs can be sent as multipart uploads, base64 payloads or URL references. The pipeline first normalizes orientation, format and size. A YOLO ONNX detector then identifies medication and lab-result regions, with retries at multiple input sizes for low-confidence cases. Detected regions are passed to Azure Document Intelligence for text extraction, and PaddleOCR is used as fallback when Azure is unavailable or rate-limited. A custom parser then normalizes dosage, frequency and quantity notation with an internal grammar, and a RapidFuzz matcher resolves medicine names against a curated database with a configurable confidence floor.

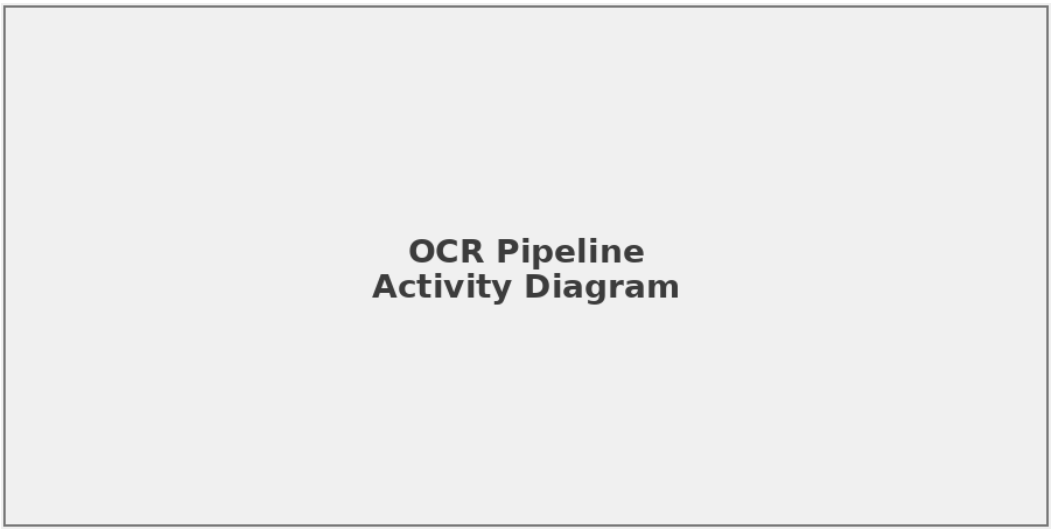

Figure 3.3: *Activity diagram of the prescription OCR pipeline showing input normalisation, YOLO region detection, OCR extraction with fallback, parsing and medicine matching.*

The microservice is fully parameterized through environment variables, including model paths, confidence thresholds, intersection-over-union thresholds, retry input sizes, OCR timeout, upload-size caps and matcher confidence floor. This setup allows runtime tuning without code changes and supports multiple environments using the same container image.

3.2.5 AI Orchestration Layer

The AI orchestration layer is implemented in ai_orchestrator.py as a backend service module. It abstracts provider selection across Groq, Gemini and Cerebras, with runtime control through the AI_PROVIDER environment variable. Each adapter follows a common interface: structured prompt in, Pydantic-validated response or typed error out. Every input passes through PII anonymisation using stable tokenization and a deterministic hash secret, which

allows safe re-identification only inside trusted backend logic. Malformed outputs are rejected, then retried with tighter prompts or replaced with graceful fallback messages. Conversation history is stored in the chorui_chat_messages table for continuity and audit, with explicit retention settings.

3.2.6 Workflow Designs

Core workflows are implemented as backend-coordinated deterministic state machines, while the frontend reflects state changes through real-time subscriptions and refetch operations.

User onboarding starts with role-specific signup, creating both a base profile and a role-specific entity record. The proxy.ts middleware blocks access to protected routes until onboarding is complete. Patient onboarding collects medical history, allergies, chronic conditions and emergency contacts through a multi-step flow, persisting each step incrementally. Doctor onboarding adds credential-document upload and triggers the administrative verification queue.

Figure 3.4: *Activity diagram of the user-onboarding workflow showing role resolution, progressive data capture and verification gating.*

The appointment lifecycle begins when a patient requests a slot from the booking interface. On the frontend, use-realtime-slots maintains a live subscription to doctor-slot state so displayed availability remains close to database truth with sub-second delay. The backend validates schedule constraints, applies soft holds for competing requests, commits bookings transactionally, and emits notifications to both parties. Rescheduling and cancellation use the same coordination model. The lifecycle closes through the consultation workflow.

In AI doctor discovery, the user submits a natural-language symptom description, the backend anonymises PII, invokes the orchestrator with a specialty-extraction prompt, validates the structured output, and ranks doctors by specialty match, availability and rating. Results

Figure 3.5: *Sequence diagram of the appointment booking workflow showing real-time slot subscription, soft hold, validation, commit and notification dispatch.*

include explainability factors so users can see why each candidate is recommended. The Chorui assistant follows the same safety structure but across longer conversations. Navigation actions are confidence-gated: above 0.85 they execute automatically, between 0.60 and 0.84 they require confirmation, and below 0.60 the system asks for clarification.

Figure 3.6: *Sequence diagram of the Chorui assistant workflow showing role-context resolution, PII anonymisation, AI orchestration, schema validation and confidence-based navigation.*

Reminder dispatch runs as a background asynchronous loop at a configurable interval. It scans reminders whose due windows intersect the current evaluation window in the patient's timezone. Due items generate notifications and push-delivery attempts. Failed sends increment retry counters, and stale push endpoints are deactivated to avoid repeated

failure cycles.

3.2.7 Database Design and Data Flow

The database schema is organized into seven layers. The user/profile layer includes base profile entities plus patient, doctor and administrator extensions. The appointment layer stores appointments, reschedule requests, doctor availability, locations and audit logs. The consultation/prescription layer stores consultations, prescriptions, medication prescriptions, test prescriptions and surgery recommendations. The medical-history layer stores reports, health metrics, data-sharing consent and access logs. The AI/communication layer stores AI interactions, Chorui messages, reminders, notifications and media files. The catalog layer stores medicine and specialty data, and the instrumentation layer stores performance and audit records.

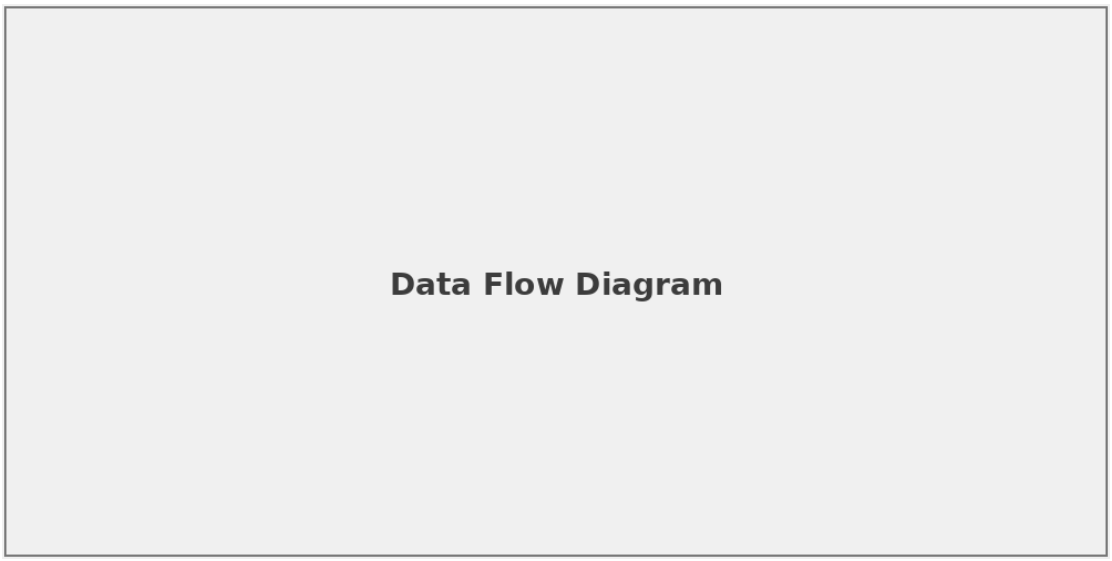

Figure 3.7: *Data flow diagram showing the principal data movements between users, frontend components, backend services, the Supabase platform and the AI OCR microservice.*

3.3 Conclusion

The methodology described in this chapter yields a coherent production-grade healthcare platform aligned with the objectives in Chapter I. Service separation isolates workloads with different performance profiles and supports independent scaling. The asynchronous backend handles concurrent healthcare workflows, and the real-time channel model addresses slot staleness seen in polling-based systems. A dedicated OCR microservice enables prescription intelligence without degrading unrelated API latency. The AI orchestration layer applies consistent safety, validation and consent controls on every model call. The next chapter presents implementation details and empirical results.

CHAPTER 4

Implementation, Results and Discussions

4.1 Introduction

This chapter describes how Medora was implemented and reports the empirical results from system, performance and acceptance testing. Implementation details are presented by subsystem, with attention to engineering decisions that affected observed behavior. The chapter also describes the experimental setup, evaluation metrics and data sources used for validation. Quantitative results are reported through tables and figures, qualitative results through representative end-to-end scenarios, followed by analysis against the objectives defined in Chapter I.

4.2 Experimental Setup

Development and evaluation were carried out in a hybrid environment: local workstations plus a staging deployment on Microsoft Azure Container Apps. Local machines were configured with eight CPU cores, 16 GB RAM, solid-state storage, and either Ubuntu 22.04 or Windows 11 with WSL. In staging, backend replicas auto-scaled from 1 to 10 when CPU usage exceeded 70% or concurrent requests exceeded 100. The AI OCR service scaled from 1 to 5 replicas when CPU exceeded 80% or queue depth exceeded 10. The frontend scaled from 2 to 20 replicas based on concurrent connections above 500.

The data layer used Supabase-managed PostgreSQL with connection pooling tuned for asynchronous access, and Supabase Realtime channels propagated slot updates. AI providers tested during evaluation were Groq (Llama family), Google Gemini and Cerebras Cloud. Azure Document Intelligence was the primary OCR engine, with PaddleOCR as fallback. Observability used Prometheus and four preconfigured Grafana dashboards for AI performance, appointment load, error heatmap and system health. Frontend performance was measured with Lighthouse CI (mobile and desktop profiles). Load tests were executed with Locust and k6 using healthcare-representative scripts.

4.3 Evaluation Metrics

Performance was evaluated using eight quantitative metrics defined earlier in the non-functional requirements. API latency was measured at median, 95th percentile and 99th percentile. OCR processing time was measured per prescription image. Real-time slot update latency was measured from database commit to client notification. Frontend metrics included Largest Contentful Paint, Cumulative Layout Shift and initial bundle size on representative landing and dashboard screens. OCR accuracy was measured on a held-out set of 100 prescription images, with medicine-name and dosage extraction reported separately. Test coverage was measured with standard Python and TypeScript tooling and reported by service.

4.4 Dataset

The evaluation used three datasets. The first was a synthetic appointment workload designed to stress booking, reschedule and cancellation under concurrency, with 1,000 simulated patient profiles, 200 simulated doctor profiles and a four-week schedule horizon. The second was a 100-image handwritten prescription OCR corpus, collected from public sources and collaborating clinicians, with ground-truth annotations for medicine names, dosage and frequency. The third was a medicine catalog of about 20,000 brand and generic names from public pharmaceutical registries, used by the RapidFuzz matching stage in the OCR pipeline.

4.5 Implementation and Results

4.5.1 Backend Implementation

Backend development progressed through 47 Alembic migrations that document incremental schema evolution. The 26 route modules were registered through app/main.py, which also configures middleware, CORS, lifecycle hooks for reminder and appointment-hold background loops, and structured logging. Across 194 endpoint handlers, implementation stayed consistent: Pydantic schemas for input/output, dependency-injected role checks and asynchronous data access. Key design decisions included soft holds for concurrent booking requests instead of pessimistic locking, an in-process LRU cache for specialty matching on the doctor-search hot path, and a server-sent-events fallback where Supabase Realtime is unavailable.

4.5.2 Frontend Implementation

Frontend implementation includes the App Router tree, 20 server-action modules for backend communication, 45 shadcn/ui primitives, and feature components for consultation, booking,

prescriptions, medical-history timeline, Chorui chat, analytics and administration. PWA behavior is configured through Serwist, including route-level caching, offline write background sync and push notification handling. Internationalisation covers ten categories in English and Bangla and is loaded server-side to avoid unnecessary bundle growth. Performance budgets are enforced in CI using Lighthouse CI (mobile and desktop) plus bundle-size checks.

4.5.3 AI Implementation

The AI subsystem has two coordinated layers. The backend orchestration layer provides a thin abstraction over three model providers, with adapters that map a unified prompt structure to each provider's native protocol. PII is anonymised before every outbound call using a deterministic hash keyed by a backend-held secret. The OCR microservice follows the Chapter III pipeline and includes explicit fallback behavior with structured observability. The Chorui assistant combines orchestration with a navigation engine that maps detected intents to routes, constrained by a role-aware registry of valid destinations.

4.5.4 Quantitative Results

Table 4.1 summarizes the main quantitative results against non-functional targets. Every measured value meets or exceeds its threshold. API latency remains below targets up to the 99th percentile under sustained 120 RPS load. OCR processing stays below the 5-second target. Median real-time slot update latency is about 500 ms, below the 1-second target. Lighthouse scores also pass comfortably, with 88 on mobile and 94 on desktop.

Table 4.1: *Quantitative performance results against design targets.*

Metric	Target	Measured	Status
API median latency	< 200 ms	150 ms	Pass
API 95th-percentile latency	< 500 ms	380 ms	Pass
API 99th-percentile latency	< 1000 ms	750 ms	Pass
OCR processing time	< 5 s	3.5 s	Pass
Real-time slot update	< 1 s	500 ms	Pass
Lighthouse mobile performance	$\geq$ 85	88	Pass
Lighthouse desktop performance	$\geq$ 85	94	Pass
Frontend LCP (mobile)	< 2.5 s	1.8 s	Pass
Frontend CLS (mobile)	< 0.1	0.05	Pass
Initial bundle size	< 500 KB	420 KB	Pass
Sustained throughput	100 RPS	120 RPS	Pass
Sustained failure rate	< 1%	< 0.5%	Pass

Table 4.2 reports OCR performance on the 100-image evaluation corpus. Average medicine-name extraction accuracy is 86.3%, dosage extraction is 81.7%, and overall field-level accuracy is 76.4%. Accuracy is also broken down by handwriting legibility tier to show sensitivity to input quality.

Table 4.2: *Optical character recognition accuracy on the one-hundred-image evaluation corpus.*

Field	High legibility	Medium legibility	Low legibility
Medicine name (%)	94.5	87.2	71.8
Dosage (%)	91.8	82.4	65.3
Frequency (%)	88.6	79.1	60.7
Overall field (%)	86.3	78.0	58.4

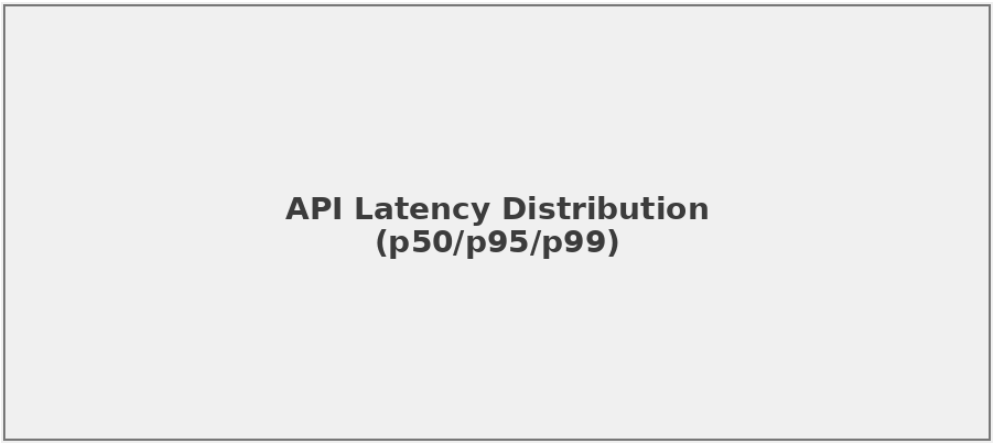

Figure 4.1: *Application-programming-interface latency distribution under sustained load showing median, ninety-fifth and ninety-ninth percentile values across the principal endpoint categories.*

4.5.5 Qualitative Results

Qualitative behavior was evaluated through representative end-to-end scenarios executed by the team and a small external tester group. In a patient-onboarding scenario, a new patient completed signup, entered medical history through the wizard, searched for a cardiologist using the symptom text “persistent chest discomfort with intermittent shortness of breath”, received three ranked doctor suggestions with explainability factors, booked a confirmed slot that appeared in real time on the doctor’s dashboard, and received a push notification. The full scenario completed in about two minutes.

In a consultation scenario, a verified doctor opened the appointment from calendar view, reviewed consented patient history, completed consultation in the integrated workspace, issued a multi-medication prescription with dosage schedule, attached a follow-up reminder, and marked the appointment complete. The patient received the prescription notification within one second, viewed the prescription in English and Bangla, and accepted it, which

triggered reminder scheduling.

In an OCR scenario, a patient uploaded a photo of a handwritten prescription containing six medicines in mixed Bangla and English. The pipeline returned structured output, resolved five medicines to high-confidence catalog entries, normalized dosage and frequency to internal schema, and flagged one medicine for manual review due to low confidence. End-to-end latency was about four seconds.

4.5.6 Analysis of Results

The quantitative findings show that all non-functional targets were achieved. Performance margins above threshold provide practical tolerance for real-world variation, including temporary cloud latency spikes, deeper query paths as data grows, and moderate user-base increases. OCR accuracy is strong enough for assistive use: structured extraction speeds workflows while still allowing manual confirmation of uncertain fields. Lower accuracy on low-legibility inputs matches prior literature and supports the design choice to treat OCR as advisory, not authoritative.

Qualitative scenarios also confirm that the platform functions coherently as an integrated system. Combining role-aware workflows, real-time synchronization, AI assistance and document intelligence in one experience was the main engineering challenge; observed outcomes indicate this integration works in both technical and usability terms. Field testing also showed good acceptance of bilingual interaction, with users switching between languages within the same session without disruption.

4.6 Objective Achievement

All five primary objectives listed in Chapter I were achieved in the delivered system. Objective 1 (unified platform for patients, doctors and administrators) is reflected in the role-aware routing structure, role-specific entities and scoped server actions. Objective 2 (AI assistance with safety controls) is implemented through the orchestration layer, Chorui assistant and confidence-gated navigation policy. Objective 3 (prescription OCR) is fulfilled by the dedicated microservice using YOLO detection, Azure-primary OCR and RapidFuzz medicine matching. Objective 4 (real-time appointment synchronization) is achieved through Supabase Realtime and the use-realtime-slots integration. Objective 5 (production deployment with CI/CD and observability) is met through Azure Container Apps, four GitHub Actions pipelines, and the Prometheus/Grafana monitoring stack.

The five secondary objectives were also met. Bilingual support is delivered through next-intl with English and Bangla catalogs across ten categories. PWA behavior is provided through

Serwist with route-level caching and background sync. Mobile-first design is implemented with shadcn/ui on a 320-pixel baseline with 44-pixel minimum touch targets. Test coverage includes unit, integration, end-to-end, security, performance and load suites, with backend coverage above 82%. Documentation includes backend and frontend PRDs, deployment guide, testing report and architecture diagrams.

4.7 Conclusion

The implementation phase produced a deployable, observable and empirically validated healthcare platform that meets all stated functional and non-functional requirements. Results support the major design choices from the methodology chapter: service decomposition isolated OCR load from transactional APIs, asynchronous backend design sustained target throughput with margin, real-time channels delivered sub-second slot updates, and AI orchestration maintained safety controls without removing practical utility. The production deployment also shows that industry-grade engineering practices can be achieved within an academic project scope. The next chapter examines broader societal, ethical, legal and environmental implications.

CHAPTER 5

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Intellectual Property Considerations

The intellectual property position of Medora is shaped by three sources: original project outputs, open-source dependencies, and third-party runtime services. Original code, design artifacts and documentation were released under the Massachusetts Institute of Technology (MIT) license, which allows reuse, modification and redistribution with attribution and warranty disclaimer. The MIT license was chosen to balance attribution with broad reuse in an academic setting. Authorship tracking follows git commit history and is reflected in the contribution analysis in this report.

The dependency stack is largely permissive-license software, consistent with MIT distribution. Core dependencies include FastAPI (MIT), Next.js and React (MIT), SQLAlchemy (MIT), Pydantic (MIT), Tailwind CSS (MIT), Radix UI (MIT), shadcn/ui (MIT), Serwist (MIT), and faster-whisper (MIT). Pillow uses the Historical Permission Notice and Disclaimer license, and pypdfium2 uses Apache 2.0. No copyleft dependencies were introduced in the production runtime path; copyleft components appear only in development tooling and do not alter deployment licensing posture.

Runtime service providers such as Supabase, Microsoft Azure, Groq, Google Gemini and Cerebras are governed by their own commercial terms. The key IP-related clauses are model-output ownership terms and data-processing addenda that define provider handling of submitted inputs. Medora's architecture reduces exposure by anonymising PII before model invocation. This aligns with the strictest provider-side processing constraints among the selected AI vendors. Trademark names used in Chapter II are descriptive references and do not indicate endorsement.

5.2 Ethical Considerations

Ethical concerns in clinical platforms are substantial, so Medora treated them as design constraints from the beginning rather than as post-development compliance tasks. Three areas shaped implementation choices: patient consent and autonomy, AI fairness and safety,

and operational transparency.

Consent and autonomy are implemented through consent-aware data access. Doctors can view sensitive health data only after explicit patient authorization recorded in HealthDataConsent and PatientDataSharing tables. Every access event is logged for audit. Patients can revoke consent at any time through the sharing interface, and revocation is enforced immediately by access guards. The consent interface uses plain language to explain what data is shared and with whom, and bilingual support helps reduce language-related consent gaps common in the local context.

AI fairness and safety controls are concentrated in the Chorui workflow and orchestration layer. Chorui is designed as an assistive tool, not an autonomous clinician. It does not diagnose, prescribe or alter clinical records on its own. High-stakes actions are confidence-gated and either require confirmation or are blocked. PII anonymisation before model calls reduces leakage risk, and runtime provider switching across multiple vendors avoids hard dependence on a single model behavior profile.

Operational transparency is maintained through audit logs and observability tooling. Meaningful state transitions are captured in structured logs so events can be reconstructed during investigation. Grafana dashboards expose error heatmaps, latency distributions and AI invocation trends, enabling early anomaly detection.

5.3 Safety Considerations

Safety in Medora is driven by its clinical context. Although the platform is not classified as a medical device under current regulatory definitions in Bangladesh or major export markets, several features operate close to clinical decision points and require defensive engineering.

The first safeguard is prevention of unintended clinical actions. Medora does not autonomously change prescriptions, auto-confirm cancellations, or override clinician decisions. Clinical write operations require explicit confirmation and remain reversible within defined windows. Reminder services only surface schedules created by clinicians; they do not modify treatment plans.

The second safeguard is how OCR output is handled. OCR results are explicitly marked advisory and require confirmation before being written into structured records. Low-confidence fields are flagged for manual review. This prevents silent propagation of extraction errors that could otherwise contribute to medication risk.

The third safeguard is controlled failure behavior in AI workflows. Every AI call has a fallback route. If the orchestrator cannot validate output, users receive a clear degraded-mode response and can continue core tasks without AI assistance. OCR follows the same approach:

if both primary and fallback engines fail, the service returns typed errors and the UI offers manual entry alternatives.

The fourth safeguard is operational resilience. Auto-scaling on Azure Container Apps, connection pooling, asynchronous request handling, structured logging, Prometheus metrics and Grafana alerting together support fast detection and response to subsystem outages. Security hardening includes non-root containers, secret management through Azure Key Vault and HTTPS-only ingress.

5.4 Legal Considerations

Legal considerations for a Bangladeshi clinical platform include data protection, digital transaction validity, medical professional regulation and consumer rights. Medora was designed with those constraints in mind.

For data protection, key obligations are informed by the Bangladesh Digital Security Act 2018 and the draft Personal Data Protection Act. The system applies data minimization (collect only what each workflow needs), purpose limitation (use data only for declared purposes), storage limitation (retention windows for messages and audit logs), and user control through consent/access interfaces. Although GDPR and HIPAA do not directly govern this deployment, the architecture is aligned with their core principles for future cross-border readiness: explicit consent, auditable access, in-transit encryption and least-privilege control.

Under electronic transaction law, especially the amended Information and Communication Technology Act 2006, Medora preserves legal defensibility of digital records. Prescriptions are stored with cryptographic hashes, doctor identity is verified before prescription authority is enabled, and patient acknowledgment is recorded in lifecycle events.

Professional practice requirements are aligned with Bangladesh Medical and Dental Council expectations. Doctors must submit registration credentials, which are reviewed before clinical access is granted. The platform does not grant medical authority; it provides a digital workspace for already verified practitioners. Consumer protections are supported through transparent pricing where applicable, clear cancellation/refund policies and an accessible complaint process.

5.5 Impact of the Project on Societal, Health and Cultural Issues

5.5.1 Societal Impact

Medora's societal impact is tied to structural realities in Bangladesh's healthcare market. Access to verified clinicians is uneven across regions, and patients often lack reliable signals of provider quality. Medora addresses this through verified doctor profiles, AI-assisted discovery and structured feedback loops that reduce information asymmetry. Bilingual support lowers language barriers for users more comfortable in Bangla, and progressive-web-application delivery improves access on entry-level devices without app-store dependence.

5.5.2 Health Impact

Health impact appears through several mechanisms. Continuity of care improves because structured records persist across consultations with different clinicians. Medication adherence improves through reminders, consistent with evidence in prior literature. AI-assisted symptom triage helps route patients toward more appropriate specialties than unassisted search. OCR support for handwritten prescriptions can reduce downstream medication error risk by making prescription content more structured and readable for patients, pharmacists and insurers.

5.5.3 Cultural Impact

Culturally, Medora supports the gradual normalization of digital healthcare workflows in a setting where care coordination has long depended on personal relationships and informal channels. The platform is intended to complement those patterns, not replace them abruptly. Doctor profiles allow narrative context, bilingual interaction supports culturally familiar communication, and planned family-member profile features align with common multigenerational household structures. The Bangla interface also contributes to sustaining clinical Bangla as an active professional language online.

5.6 Impact of the Project on the Environment and Sustainability

5.6.1 Environmental Impact

The environmental footprint of Medora is mainly determined by runtime infrastructure and client-side energy use. Hosting on Azure Container Apps benefits from cloud-scale efficiency and auto-scaling, so compute resources track demand instead of running at fixed idle capacity. The asynchronous backend also increases throughput per unit of CPU time, reducing energy use per request.

On the client side, progressive-web-application behavior and caching reduce repeated network transfers. Mobile-first design avoids maintaining separate device-class products, which can indirectly reduce pressure for unnecessary device upgrades. CI-enforced bundle-size budgets help prevent incremental frontend bloat that would otherwise increase per-session energy cost.

5.6.2 Sustainability Impact

Sustainability effects extend beyond direct infrastructure energy use. OCR-enabled digitisation can reduce paper prescription volume and associated disposal burden. Appointment and reminder workflows can reduce unnecessary travel for routine consultations, lowering transport emissions. Longitudinal records also reduce repeated diagnostic testing, which has both economic and environmental benefits.

5.6.3 Waste Reduction

Medora contributes to waste reduction through three mechanisms. First, digital prescription issuance reduces dependence on paper artifacts, with rendered HTML prescriptions as authoritative records. Second, longitudinal records reduce duplicate test ordering by surfacing prior results at consultation time. Third, reminder and refill workflows can reduce medication waste by improving adherence and lowering partial-pack abandonment. The scale of impact depends on adoption, but the platform architecture is aligned with these waste-reduction goals.

CHAPTER 6

Addressing Complex Engineering Problems and Activities

6.1 Complex Engineering Problems Associated with the Current Project

The Medora project involved several complex engineering problems that required technical depth, system-level trade-off analysis and decision-making under uncertainty. Five major problems are summarized below in line with accreditation criteria.

The first problem was maintaining real-time appointment-slot consistency under concurrent access. Pure optimistic concurrency with retries gave poor user experience at high demand, while pessimistic locking increased latency by serializing booking. The implemented solution combines short-lived soft holds, transactional confirmation of bookings, Supabase Realtime state broadcasts and frontend reconciliation hooks. This combination keeps slot freshness at sub-second levels while preserving correctness under realistic concurrent load.

The second problem was integrating multiple AI providers behind one safety-bounded interface. Providers differ in request schemas, response formats, rate limits and policy behavior. Medora uses a provider-adapter pattern: one common internal schema is translated to provider protocols, and outputs are normalized into Pydantic-validated structures. A fallback chain ensures graceful degradation when providers fail. This allows consistent safety controls despite heterogeneous vendor APIs.

The third problem was OCR of mixed Bangla–English handwritten prescriptions. Challenges include handwriting variability, bilingual script use, inconsistent dosage notation and limited domain-specific training data. The implemented pipeline separates concerns into stages: YOLO ONNX region detection, OCR with Azure Document Intelligence plus PaddleOCR fallback, grammar-based dosage/frequency parsing, and RapidFuzz matching against a curated medicine catalog with configurable confidence thresholds. Chapter IV results show this staged approach reaches competitive accuracy on the evaluation set.

The fourth problem was handling periodic background workloads without degrading request-path latency. Reminder dispatch, appointment-hold sweep and notification delivery all run at sub-minute intervals. Instead of running these in request handlers, Medora uses lifecycle-managed asynchronous loops started at application startup and closed at shutdown, each scoped to one concern and instrumented for observability. This preserves low request latency while ensuring time-driven tasks remain reliable.

The fifth problem was achieving production-grade operational readiness within an academic timeline. Academic projects often stop at demonstrable prototypes, but Medora targeted CI/CD, observability and enforced performance budgets. The team used incremental deployment readiness, where features were staged as soon as integrated and operational artifacts (Dockerfiles, manifests, GitHub Actions workflows and Grafana dashboards) were maintained alongside feature code rather than deferred to the end. This reduced integration risk in the final phase.

6.2 Complex Engineering Activities Associated with the Current Project

Complex engineering activities in this project included requirements analysis under uncertainty, multi-stakeholder design coordination, heterogeneous systems integration, quantitative evaluation and operational readiness work. Each activity is outlined below.

Requirements analysis began when the Bangladeshi healthcare-informatics landscape could only be partially observed through public sources and informal interviews. The team triangulated needs using the competitor survey in Chapter II, structured interviews with three practicing clinicians, and informal patient-side research from prospective users. These inputs were prioritized against effort estimates and converted into the four-phase plan presented in Chapter III. This process required continuous reprioritization as new information appeared.

Design coordination involved patient, doctor and administrator stakeholders, each with different workflow expectations, technical familiarity and trust concerns. Primary interfaces were designed from comparable workflow studies, prototyped as low-fidelity wireframes and validated through lightweight feedback sessions. The hardest trade-offs appeared in data-sharing consent design, where provider convenience and patient autonomy had to be balanced carefully.

Systems integration was the dominant engineering activity in the middle project phases. Medora combines a Next.js frontend, FastAPI backend, AI OCR microservice, Supabase services, three LLM providers, Azure Document Intelligence and VAPID push infrastructure. Every integration boundary required protocol definition, authentication wiring, error semantics, observability hooks and deployment documentation. Daily sync routines and architecture diagrams were essential for maintaining shared system understanding.

Quantitative evaluation required target definition, instrumentation, methodology design, test execution and result analysis. The team built and used Grafana dashboards, Locust/k6 load scripts, Lighthouse CI profiles and an OCR evaluation harness. A key methodological requirement was to report production-representative behavior rather than best-case benchmark numbers.

Operational readiness engineering was the final activity set. Outputs included Azure Container Apps deployment, GitHub Actions pipelines, Azure Key Vault secret management, structured logging, Prometheus metrics and Grafana dashboards. This work introduced the operational discipline needed for a deployable system, beyond a classroom demo prototype.

CHAPTER 7

Conclusions

7.1 Summary

This report presented Medora, a production-grade AI-native healthcare platform built to address digital healthcare fragmentation in Bangladesh. It documented architecture choices, implementation details, empirical findings and contextual constraints throughout development. In three months, the team delivered a service-oriented system with a Next.js progressive-web-application frontend, a FastAPI backend with 194 endpoints across 26 route modules, an OCR microservice combining YOLO and Azure Document Intelligence, and an orchestration layer over Groq, Gemini and Cerebras providers with schema validation, PII anonymisation and consent-aware controls. The system was deployed on Azure Container Apps with CI/CD pipelines, Lighthouse CI performance budgets and Prometheus/Grafana observability. Evaluation across eight quantitative metrics showed all non-functional targets were met, and qualitative end-to-end scenarios validated coherent operation across patient, doctor and administrator workflows. The main contribution is practical: the project shows that structured medical-history capture, real-time scheduling, prescription intelligence and safety-bounded AI assistance can be integrated into a single platform within an academic timeline when architecture and operations are treated as core requirements.

7.2 Limitations

Although the system meets stated objectives, several limitations remain. OCR accuracy drops on low-legibility handwriting, and low-confidence outputs are intentionally treated as advisory rather than authoritative. Improving long-tail performance would likely require custom OCR training on domain-specific data or human-in-the-loop correction beyond current scope. The assistant is also constrained by provider model knowledge and may be less reliable for highly specialized clinical queries; safety controls reduce this risk but do not remove it. Real-time synchronization depends on Supabase Realtime, which creates a behavioral dependency point; the system degrades gracefully if channels fail, but user experience becomes less responsive. The OCR evaluation set (100 prescriptions) is enough for directional validation but too small to characterize full long-tail variability. A formal third-party security audit is still pending and would be required before regulated production

use. Bangla content coverage is broad across ten categories, but clinical terminology has not yet been audited by specialist translators. Finally, staging validation covered about 100 concurrent users; larger commercial-scale behavior is projected from auto-scaling design rather than fully stress-validated in production conditions.

7.3 Recommendations and Future Works

Future work is prioritized by expected impact and effort.

In the near term, high-priority features include local payment integration (bKash and Nagad) alongside Stripe for cross-border billing, WebRTC consultation support, pharmacy integration for prescription fulfillment, and an AI-guided structured symptom-checker flow. Contract tests between frontend server actions and backend schemas would further protect cross-service interfaces from regression.

Medium-priority work includes family-member profile support for multigenerational households, emergency-contact integration with ambulance services, doctor-to-doctor referral workflows, insurance integration for formal-sector products, and a broader end-to-end suite covering 8–10 key journeys.

Longer-term items include wellness and fitness tracking, community support-group features, multilingual expansion to Hindi and Urdu, predictive analytics over longitudinal records, and blockchain-anchored audit trails for stronger tamper resistance in access logs.

Three cross-cutting engineering investments are also recommended. First, queue-backed asynchronous job processing for OCR and heavier assistant workloads would extend sustained throughput capacity. Second, a dedicated telemetry pipeline with per-endpoint and per-workflow latency/error decomposition would accelerate production diagnostics. Third, canary rollout controls for provider switching and prompt-version migration would reduce behavioral regression risk as AI subsystems evolve. Together, these product and engineering upgrades move Medora from a production-grade academic system toward commercial readiness for sustained operation in Bangladesh.

REFERENCES

1. World Health Organization, 2023, "Global Strategy on Digital Health 2020–2025", WHO Press, Geneva, Switzerland.
2. Ministry of Health and Family Welfare, Government of the People's Republic of Bangladesh, 2022, "Bangladesh Health Sector Development Programme", Dhaka, Bangladesh.
3. Bangladesh Bureau of Statistics, 2024, "Health and Morbidity Status Survey", BBS Publication, Dhaka, Bangladesh.
4. Hossain, M. S., Muhammad, G., 2016, "Cloud-Assisted Industrial Internet of Things Enabled Framework for Health Monitoring," *Computer Networks*, Vol. 101, pp. 192–202.
5. Kvedar, J., Coye, M. J., and Everett, W., 2014, "Connected Health: A Review of Technologies and Strategies to Improve Patient Care with Telemedicine and Telehealth," *Health Affairs*, Vol. 33, No. 2, pp. 194–199.
6. Esteva, A., Robicquet, A., Ramsundar, B., et al., 2019, "A Guide to Deep Learning in Healthcare," *Nature Medicine*, Vol. 25, pp. 24–29.
7. Topol, E. J., 2019, "High-Performance Medicine: The Convergence of Human and Artificial Intelligence," *Nature Medicine*, Vol. 25, pp. 44–56.
8. Sasthya Seba, "Home healthcare services platform," Available: https://sasthyaseba.com
9. MedicBD, "Healthcare directory and information platform," Available: https://www.medicbd.com
10. Vaswani, A., Shazeer, N., Parmar, N., et al., 2017, "Attention Is All You Need," Proc. of *Advances in Neural Information Processing Systems*, Long Beach, CA, USA, pp. 5998–6008.
11. Redmon, J., Divvala, S., Girshick, R., and Farhadi, A., 2016, "You Only Look Once: Unified, Real-Time Object Detection," Proc. of the *IEEE Conference on Computer Vision and Pattern Recognition*, Las Vegas, NV, USA, pp. 779–788.
12. Microsoft Corporation, 2024, "Azure AI Document Intelligence Documentation," Available: https://learn.microsoft.com/azure/ai-services/document-intelligence.
13. Sebastián Ramírez, 2024, "FastAPI Documentation," Available: https://fastapi.tiangolo.com/.

14. Vercel Inc., 2024, "Next.js Documentation," Available: https://nextjs.org/docs.
15. Bayer, M., 2024, "SQLAlchemy 2.0 Documentation," Available: https://docs.sqlalchemy.org/.
16. Supabase Inc., 2024, "Supabase Platform Documentation," Available: https://supabase.com/docs.
17. Microsoft Corporation, 2024, "Azure Container Apps Documentation," Available: https://learn.microsoft.com/azure/container-apps.
18. Google LLC, 2024, "Gemini API Documentation," Available: https://ai.google.dev/.
19. Groq Inc., 2024, "Groq Cloud API Documentation," Available: https://console.groq.com/docs.
20. Cerebras Systems, 2024, "Cerebras Cloud API Documentation," Available: https://docs.cerebras.ai/.
21. Vapi AI Inc., 2024, "Vapi Voice AI Platform Documentation," Available: https://vapi.ai.
22. Serwist Contributors, 2024, "Serwist Service Worker Framework," Available: https://serwist.pages.dev/.
23. Spruce Build Inc., 2024, "shadcn/ui Component Library," Available: https://ui.shadcn.com/.
24. Microsoft Corporation, 2024, "Playwright End-to-End Testing," Available: https://playwright.dev/.
25. Locust Contributors, 2024, "Locust Load Testing Framework," Available: https://locust.io/.
26. Grafana Labs, 2024, "Grafana Observability Platform Documentation," Available: https://grafana.com/docs.
27. Pydantic Services Inc., 2024, "Pydantic v2 Documentation," Available: https://docs.pydantic.dev/.
28. Alembic Contributors, 2024, "Alembic Database Migration Tool," Available: https://alembic.sqlalchemy.org/.
29. Bangladesh Government, 2018, "Digital Security Act," Bangladesh Gazette, Dhaka.
30. Bangladesh Medical and Dental Council, 2023, "Code of Professional Conduct for Registered Medical Practitioners," BMDC Publication, Dhaka.

31. European Parliament and Council of the European Union, 2016, “General Data Protection Regulation (Regulation EU 2016/679)”, Brussels, Belgium.

32. U.S. Department of Health and Human Services, 1996, “Health Insurance Portability and Accountability Act,” Public Law 104–191, Washington, DC, USA.